

Programación de Servicios y Procesos

3.3 Sockets UDP

3.3. Sockets UDP

3.3.1. Comunicación cliente/servidor con sockets UDP

DatagramSocket

- DatagramPacket
- Programación de aplicaciones Cliente y/o Servidor
- 3.3.2. Cliente UDP
- 3.3.3. Servidor UDP
- 3.3.4. Multicast socket

3.3.1. Comunicación cliente/servidor con sockets UDP

Igual que en el apartado anterior, Oracle proporciona una guía con información básica sobre el uso de los Sockets UDP. De nuevo, todo lo que podemos ver en ese tutorial lo vamos a ir comentando y ampliando en este apartado del tema

Tutorial de Oracle: All about datagrams (<https://docs.oracle.com/javase/tutorial/networking/datagrams/index.html>)



UDP - Protocolo sin conexión

El protocolo de comunicaciones con `datagramas` UDP, es un protocolo sin conexión, es decir, cada vez que se envíen datagramas es necesario enviar el descriptor del socket local y la dirección del socket que debe recibir el datagrama. Como se puede ver, hay que enviar datos adicionales cada vez que se realice una comunicación.

Se trata de un servicio de transporte sin conexión. Son más eficientes que TCP, pero no está garantizada la fiabilidad: los datos se envían y reciben en paquetes, cuya entrega no está garantizada; los paquetes pueden ser duplicados, perdidos o llegar en un orden diferente al que se envió.

La interfaz Java que da soporte a sockets TCP está constituida por las clases `DatagramPacket` y `DatagramSocket`.

- `DatagramSocket`: es la clase utilizada para realizar el envío y la recepción de los datos. A diferencia de los sockets TCP, esta clase no es la encargada de gestionar las direcciones ni de realizar la conexión, sólo se encarga de transportar los datos del origen al destino.

Lo único que se hace es enviar los datos, mediante la creación de un socket y utilizando los métodos de envío y recepción apropiados.

Esta clase proporciona los métodos `send` y `receive`.

- `DatagramPackets`: esta clase es la encargada de incluir la información que se quiere enviar/recibir y la información de direccionamiento, es decir, la dirección a la que se quiere enviar la información que contiene.

`DatagramPacket` contiene la información relevante. Cuando se desea recibir un datagrama, éste deberá almacenarse bien en un buffer o un array de bytes. Y cuando preparamos un datagrama para ser enviado, el `DatagramPacket` no sólo debe tener la información, sino que además debe tener la dirección IP y el puerto de destino.



Puertos duplicados UDP / TCP

Dado que la gestión de los puertos y el protocolo que se utiliza es diferente, podemos usar el mismo número de puerto para un servicio que use el protocolo TCP y otro servicio, en el mismo puerto, que use UDP.

En realidad, un socket además de IP_origen, Puerto_origen, IP_destino, Puerto_destino, también incluye el protocolo usado, por eso un socket con el mismo origen y destino, es diferente y puede ser usado a la vez por UDP y TCP

Socket = [Protocolo (TCP/UDP) + IP_origen + Puerto_origen + IP_destino + Puerto_destino]

DatagramSocket

Esta clase proporciona los siguientes métodos

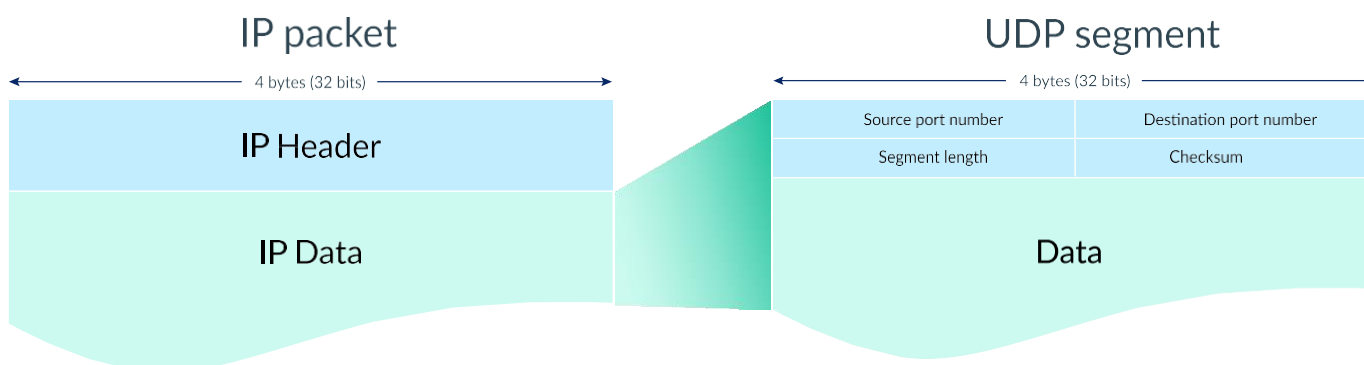
Método	Descripción
<code>public DatagramSocket () throws SocketException</code>	Se encarga de construir un socket para datagramas y de conectarlo al primer puerto disponible.
<code>public DatagramSocket (int port) throws SocketException</code>	Ídem, pero con la salvedad de que permite especificar el número de puerto asociado.
<code>public DatagramSocket (int port, InetAddress ip) throws SocketException</code>	Permite especificar, además del puerto, la dirección local a la que se va a asociar el socket.
<code>public int getLocalPort()</code>	Retorna el número de puerto en el host local al que está conectado el socket.
<code>public void receive (DatagramPacket p) throws IOException</code>	Recibe un DatagramPacket del socket, y llena el buffer con los datos que recibe.
<code>public void send (DatagramPacket p) throws IOException</code>	Envía un DatagramPacket a través del socket.
<code>public setSoTimeout(int timeout)</code>	Permite establecer un tiempo de espera límite para que el método receive se quede bloqueado esperando a recibir una respuesta por parte del otro extremo. Si no reciben datos en el tiempo fijado se lanza la excepción <code>InterruptedIOException</code>

El DatagramSocket, cuando se utiliza en la parte receptora (la que vamos a llamar servidora) que ofrece el servicio para que los clientes se conecten, sólo va a indicar el puerto al que esos clientes deben enviar sus solicitudes. En el caso de los procesos que actúen como clientes, se usará el constructor sin parámetros para que sea el SO el que asigne un puerto libre.

Por lo tanto, un mismo DatagramSocket al no incluir ninguna información de direccionamiento puede ser reutilizado para enviar y/o recibir datagramas a/desde diferentes destinos.

DatagramPacket

La clase `DatagramPacket` cómo se ha indicado anteriormente, es un contenedor del mensaje y del destino de ese mensaje.



Esta clase proporciona los siguientes métodos

Método	Descripción
<code>public DatagramPacket(byte ibuf[], int ilength)</code>	Implementa un <code>DatagramPacket</code> para la recepción de paquetes de longitud <code>ilength</code> , siendo el valor de este parámetro menor o igual que <code>ibuf.length</code> .
<code>public DatagramPacket(byte ibuf[], int ilength, InetAddress iaddr, int iport)</code>	Implementa un <code>DatagramPacket</code> para el envío de paquetes de longitud <code>ilength</code> al número de puerto especificado en el parámetro <code>iport</code> , del host especificado en la dirección de destino que se le pasa por medio del parámetro <code>iaddr</code> .
<code>public InetAddress getAddress ()</code>	Retorna la dirección IP del host al cual se le envía el datagrama o del que el datagrama se recibió.
<code>public byte[] getData()</code>	Retorna los datos a recibir o a enviar.
<code>public int getLength()</code>	Retorna la longitud de los datos a enviar o a recibir.
<code>public int getPort()</code>	Retorna el número de puerto de la máquina remota a la que se le va a enviar el datagrama o del que se recibió.

Como se intuye de la descripción de los métodos, la forma de crear el datagrama va a depender de si queremos enviar o recibir la información, ya que en cada uno de estas acciones tendremos que indicar dónde van dirigidos esos datos (envío) o bien esa información ya vendrá incluida en el datagrama (recepción) y podremos acceder a ella a través de los métodos getter de la clase.

Es importante hacer ver que la información debe enviarse como un array de bytes y que se recibe de la misma forma, por lo que tendremos que usar los métodos de `String` o de cualquiera de los otros tipos primitivos de Java para convertirlos a bytes.

Programación de aplicaciones Cliente y/o Servidor

En el caso de aplicaciones UDP, el protocolo de comunicación no tiene sentido a nivel de capa de transporte, ya que sólo se envían y reciben mensajes y hablamos de un `protocolo no orientado a conexión`, por lo tanto, no sirve para realizar confirmaciones o diálogos entre la parte servidora y cliente.

La parte del protocolo se delega en una capa superior, que será la encargada de gestionar la comunicación a un nivel de abstracción mayor.

De todas formas, la comunicación entre ambas partes debe seguir estando sincronizada en los que a envíos / respuestas se refiere para no dejar a la otra parte bloqueada en las lecturas.



Bloqueo de lecturas con timeout

Esta característica, también disponible para los sockets TCP, permite evitar un bloqueo infinito de un hilo a la espera de recibir datos del otro extremo del socket.

Es de especial importancia en la gestión de las comunicaciones UDP ya que, como hemos dicho, no tienen por qué seguir un protocolo preestablecido a nivel de transporte.

Con el método `setSoTimeout` de `DatagramSocket` podemos fijar un tiempo de espera máximo para la recepción de datos a través del socket.

3.3.2. Cliente UDP

Si nos centramos en la parte de comunicaciones, la forma general de implementar un cliente será:

1. El cliente creará un socket para comunicarse con el servidor. Para enviar datagramas necesita conocer su IP y el puerto por el que escucha.
2. Utilizará el método `send()` del socket para enviar la petición en forma de datagrama.
 - La información se envía en un objeto de tipo `DatagramPacket`
 - El `DatagramPacket` almacena el contenido del mensaje en un array de bytes
3. Permanece a la espera de recibir respuesta
4. El cliente recibe la respuesta del servidor mediante el método `receive()` del socket.
 - La información se recibe en un objeto de tipo `DatagramPacket`
 - El `DatagramPacket` almacena el contenido del mensaje en un array de bytes
5. Cerrar y liberar los recursos.

```
1  public class BasicUDP_Client {
2
3      public static void main(String[] argv) throws Exception {
4
5          // IP y puerto al que se envía el Datagrama
6          InetAddress destino = InetAddress.getLocalHost();
7          int port = 12345;
8
9          // Buffer para recibir el datagrama
10         byte[] buffer = new byte[1024];
11
12         // El mensaje a enviar en el Datagrama se convierte a bytes
13         String mensajeEnviado = "Enviando Saludos !!";
14         buffer = mensajeEnviado.getBytes(); //codifico String a bytes
15
16         // Se preparara el DatagramPacket que se va a enviar
17         DatagramPacket datagramaEnviado = new DatagramPacket(buffer, buffer.length, destino, port);
18         // En este caso, especificamos un puerto, aunque podríamos dejarlo para
19         // que el SO asigne uno Libre
20         DatagramSocket socket = new DatagramSocket(34567);
21
22         System.out.println("Host destino : " + destino.getHostName());
23         System.out.println("IP Destino : " + destino.getHostAddress());
24         System.out.println("Puerto local del socket: " + socket.getLocalPort());
25         System.out.println("Puerto al que envio: " + datagramaEnviado.getPort());
26
27         // Envío del Datagrama
28         socket.send(datagramaEnviado);
29
30         // Cierre y Liberación de recursos
31         socket.close();
32     }
33 }
```

java

3.3.3 Servidor UDP

En los sockets UDP no se establece conexión. A pesar de que cuando los programamos sí existen diferencias entre el servidor y el cliente, estas no son tan claras como con los sockets TCP. La funcionalidad y el código que diferencia a un servidor de un cliente está más diluido.

Podemos considerar servidor al que espera un mensaje y responde; y cliente al que inicia la comunicación.

Tanto uno como otro si desean ponerse en contacto necesitan saber en qué ordenador y en qué puerto está escuchando el otro.

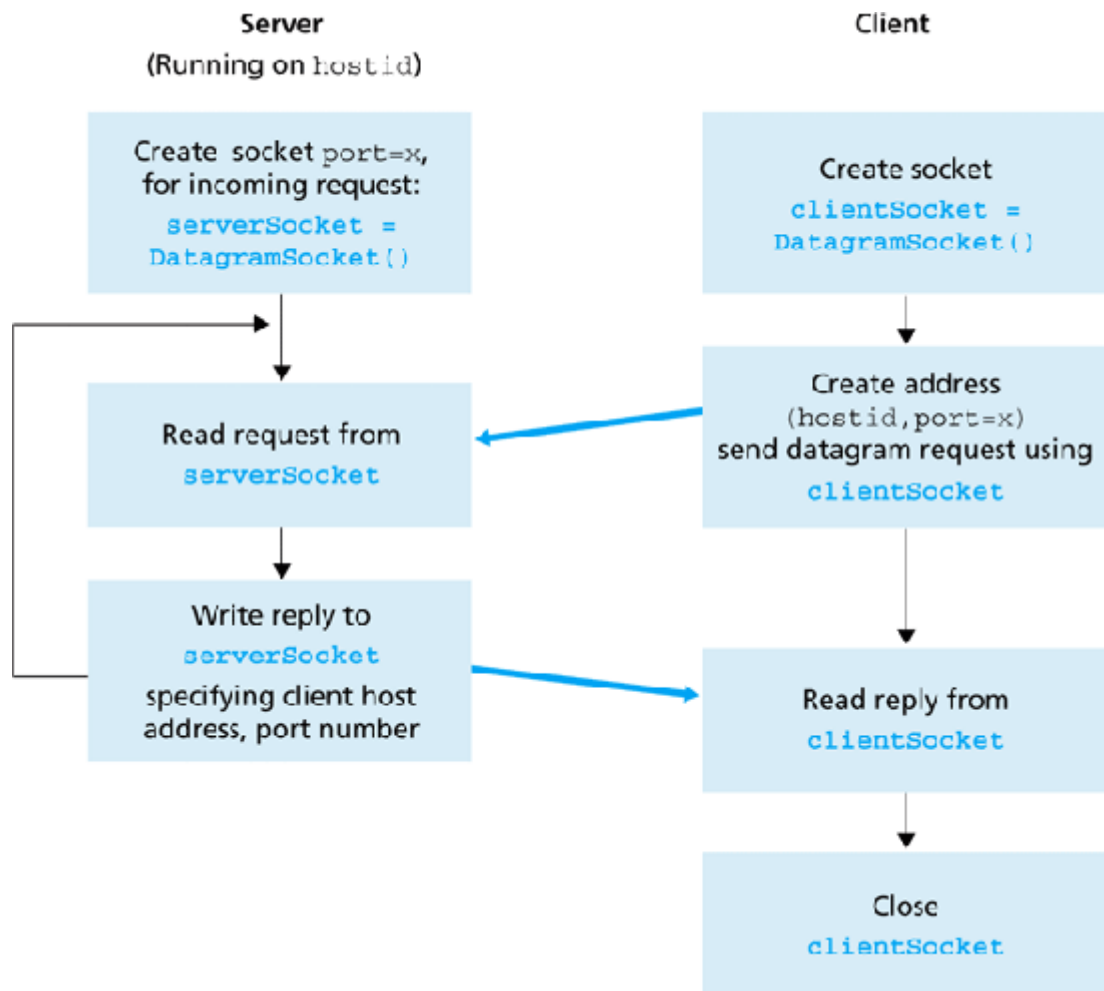
1. El servidor crea un socket asociado a un puerto local para escuchar peticiones de clientes.
2. Permanece a la espera de recibir peticiones.
3. El servidor recibe las peticiones mediante el método `receive()` del socket.
 - o La información se recibe en un objeto de tipo `DatagramPacket`
 - o El `DatagramPacket` almacena el contenido del mensaje en un array de bytes
4. En el datagrama recibido va incluido además del mensaje, el puerto y la IP del cliente emisor de la petición; lo que le permite al servidor conocer la dirección del emisor del datagrama. Utilizando el método `send()` del socket puede enviar la respuesta al cliente emisor.
5. El servidor permanece a la espera de recibir más peticiones.
6. Cerrar y liberar los recursos.

```
1  public class BasicUDP_Server {
2
3      public static void main(String[] argv) throws Exception {
4
5          // Buffer para recibir el datagrama
6          byte[] bufer = new byte[1024];
7
8          // El Socket del servidor se asocia a un puerto para que los clientes
9          // puedan enviar peticiones.
10         DatagramSocket socket = new DatagramSocket(12345);
11
12         // Se espera la llegada de un DATAGRAMA
13         // Al igual que con TCP, esta llamada a receive es bloqueante
14         // y es la que tiene que marcar la sincronización entre lecturas y
15         // escrituras de las app cliente / servidor
16         System.out.println("Esperando Datagrama.....");
17         // Se crea el objeto que almacenará el mensaje enviado por el cliente
18         DatagramPacket datagramaRecibido = new DatagramPacket(bufer, bufer.length);
19         // Se espera el mensaje y se le pasa el datagrama para que lo almacene ahí
20         socket.receive(datagramaRecibido);
21         String mensajeRecibido = new String(datagramaRecibido.getData());
22
23         // Información recibida
24         System.out.println("Número de Bytes recibidos: " + datagramaRecibido.getLength());
25         System.out.println("Contenido del Paquete : " + mensajeRecibido.trim());
26
27         System.out.println("Puerto origen del mensaje: " + datagramaRecibido.getPort());
28
29         System.out.println("IP de origen : " + datagramaRecibido.getAddress().getHostAddress());
30         System.out.println("Puerto destino del mensaje: " + socket.getLocalPort());
31
32         // Liberamos los recursos
33         socket.close();
34     }
```

java

35 }

Quedando la secuencia de acciones entre el cliente y el servidor de la siguiente manera



Veamos ahora un ejemplo completo de C/S UDP

```

1  public class BasicUDP_Client2 {
2
3      public static void main(String args[]) throws Exception {
4
5          // FLUJO PARA ENTRADA ESTANDAR
6          BufferedReader in = new BufferedReader(new InputStreamReader(System.in));
7
8          DatagramSocket clientSocket = new DatagramSocket();
9          byte[] enviados = new byte[1024];
10         byte[] recibidos = new byte[1024];
11
12         // DATOS DEL SERVIDOR al que enviar mensaje
13         InetAddress IPServidor = InetAddress.getLocalHost();// Localhost
14         int puerto = 9876; // puerto por el que escucha
15
16         // INTRODUCIR DATOS POR TECLADO
17         System.out.print("Introduce mensaje: ");
18         String cadena = in.readLine();
19         enviados = cadena.getBytes();
20
21         // ENVIANDO DATAGRAMA AL SERVIDOR
22         System.out.println("Enviando " + enviados.length + " bytes al servidor.");
23         DatagramPacket envio = new DatagramPacket(enviados, enviados.length, IPServidor, puerto);
  
```

java

```

24         clientSocket.send(envio);
25
26         // RECIBIENDO DATAGRAMA DEL SERVIDOR
27         DatagramPacket recibo = new DatagramPacket(recibidos, recibidos.length);
28         System.out.println("Esperando datagrama.....");
29         clientSocket.receive(recibo);
30         String mayuscula = new String(recibo.getData());
31
32         // OBTENIENDO INFORMACIÓN DEL DATAGRAMA
33         InetAddress IPOrigen = recibo.getAddress();
34         int puertoOrigen = recibo.getPort();
35         System.out.println("\tProcedente de: " + IPOrigen + ":" + puertoOrigen);
36         System.out.println("\tDatos: " + mayuscula.trim());
37
38         //cerrar socket
39         clientSocket.close();
40
41     }
42 }

```

```

1  public class BasicUDP_Server2 {
2
3      public static void main(String args[]) throws Exception {
4
5          //Puerto por el que escucha el servidor: 9876
6          DatagramSocket serverSocket = new DatagramSocket(9876);
7          byte[] recibidos = new byte[1024];
8          byte[] enviados = new byte[1024];
9          String cadena;
10
11         while (true) {
12             System.out.println("Esperando datagrama.....");
13
14             //RECIBO DATAGRAMA
15             recibidos = new byte[1024];
16             DatagramPacket paqRecibido = new DatagramPacket(recibidos, recibidos.length);
17             serverSocket.receive(paqRecibido);
18             cadena = new String(paqRecibido.getData());
19
20             //DIRECCION ORIGEN
21             InetAddress IPOrigen = paqRecibido.getAddress();
22             int puerto = paqRecibido.getPort();
23             System.out.println("\tOrigen: " + IPOrigen + ":" + puerto);
24             System.out.println("\tMensaje recibido: " + cadena.trim());
25
26             //CONVERTIR CADENA A MAYÚSCULA
27             String mayuscula = cadena.trim().toUpperCase();
28             enviados = mayuscula.getBytes();
29
30             //ENVIO DATAGRAMA AL CLIENTE
31             DatagramPacket paqEnviado = new DatagramPacket(enviados, enviados.length, IPOrigen, puerto);
32             serverSocket.send(paqEnviado);
33
34             // Condición de finalización
35             if (cadena.trim().equals("")) {
36                 break;
37             }
38
39             //Fin de while

```

java


```
40
41     serverSocket.close();
42     System.out.println("Socket cerrado...");
43 }
44 }
```

3.3.4 Multicast socket

La clase `MulticastSocket` es útil para enviar paquetes a múltiples destinos simultáneamente.

Para poder recibir estos paquetes es necesario establecer un grupo multicast, que es un grupo de direcciones IP que comparten el mismo número de puerto.

Cuando se envía un mensaje a un grupo de multicast, todos los que pertenezcan a ese grupo recibirán el mensaje.

La pertenencia al grupo es transparente al emisor, es decir, el emisor no conoce el número de miembros del grupo ni sus direcciones IP.

 **Grupo multicast**

Un grupo multicast se especifica mediante una dirección IP de clase D y un número de puerto UDP estándar.

Las direcciones desde la 224.0.0.0 a la 239.255.255.255 están destinadas para ser direcciones de multicast.

La dirección 224.0.0.0 está reservada y no debe ser utilizada.

Los métodos que proporciona la clase `MulticastSocket` son

Método	Descripción
<code>MulticastSocket()</code> throws <code>IOException</code>	Construye un socket multicast dejando al SO que asigne un puerto libre.
<code>MulticastSocket(int port)</code> throws <code>IOException</code>	Construye un socket multicast y lo conecta al puerto local especificado.
<code>public void receive (DatagramPacket p)</code> throws <code>IOException</code>	Recibe un <code>DatagramPacket</code> del socket, y llena el buffer con los datos que recibe.
<code>public void send (DatagramPacket p)</code> throws <code>IOException</code>	Envía un <code>DatagramPacket</code> a través del socket.
<code>joinGroup(InetAddress multicastAddress)</code>	Permite al socket unirse al grupo de multicast. A partir de ese momento podrá recibir los mensajes que se envían a esa dirección. Un <code>MulticastSocket</code> puede estar unido a más de un grupo multicast.
<code>leaveGroup(InetAddress multicastAddress)</code>	Permite al socket abandonar el grupo de multicast

Y a continuación presentamos el esquema de llamadas seguido por un **servidor multicast**

1. Se crea el socket multicast. No hace falta especificar puerto

```
MulticastSocket ms = new MulticastSocket();
```

2. Se define el puerto multicast

```
int Puerto = 12345;
```

3. Se crea el grupo multicast

```
InetAddress grupo = InetAddress.getByName("225.0.0.1");
```

4. Se crea el datagrama

```
DatagramPacket paquete = new DatagramPacket(msg.getBytes(), msg.length(), grupo, Puerto);
```

5. Se envía el paquete al grupo

```
ms.send(paquete);
```

6. Se cierra el socket

```
ms.close();
```

y el esquema de llamadas seguido por un **cliente multicast**

1. Se crea un socket multicast en el puerto establecido

```
MulticastSocket ms = new MulticastSocket(12345);
```

2. Se configura la IP del grupo al que nos conectaremos

```
InetAddress grupo = InetAddress.getByName("225.0.0.1");
```

3. Se une al grupo

```
ms.joinGroup(grupo);
```

4. Recibe el paquete del servidor multicast

```
byte[] buf = new byte[1000]; DatagramPacket recibido = new DatagramPacket(buf, buf.length); ms.receive(recibido);
```

5. Salimos del grupo multicast:

```
ms.leaveGroup(grupo);
```

6. Se cierra el socket

```
ms.close();
```

Y ahora lo vemos todo junto con un ejemplo

```
1 public class U4_BasicMulticastServer {
2
3     public static void main(String args[]) throws Exception {
4
5         // Enviamos la información introducida por teclado hasta que se envíe un *
6         Scanner in = new Scanner(System.in);
7
8         //Se crea el socket multicast.
9         MulticastSocket ms = new MulticastSocket();
10        // Se escoge un puerto para el server
11        int puerto = 12345;
12        // Se escoge una dirección para el grupo
13        InetAddress grupoMulticast = InetAddress.getByName("225.0.0.1");
14
15        String cadena = "";
16        while (!cadena.trim().equals("")) {
17
```

java

```

18         System.out.print("Datos a enviar al grupo: ");
19         cadena = in.nextLine();
20
21         // Enviamos el mensaje a todos los clientes que se hayan unido al grupo
22         DatagramPacket paquete = new DatagramPacket(cadena.getBytes(), cadena.length(), grupoMulticast, puerto);
23         ms.send(paquete);
24     }
25
26     // Cerramos recursos
27     ms.close();
28     System.out.println("Socket cerrado...");
29 }
30 }

```

```

1 public class U4_BasicMulticastClient {
2
3     public static void main(String args[]) throws Exception {
4
5         //Se crea el socket multicast
6         // El puerto debe ser el mismo en todos los clientes, ya que el
7         // servidor multicast envía la información a la IP multicast y a un puerto
8         int puerto = 12345; //Puerto multicast
9         MulticastSocket ms = new MulticastSocket(puerto);
10
11         //Nos unimos al grupo multicast
12         InetAddress grupo = InetAddress.getByName("225.0.0.1");
13         ms.joinGroup(grupo);
14         String msg = "";
15
16         while (!msg.trim().equals("")) {
17             // El buffer se crea dentro del bucle para que se sobrescriba
18             // con cada nuevo mensaje
19             byte[] buf = new byte[1000];
20             DatagramPacket paquete = new DatagramPacket(buf, buf.length);
21             //Recibe el paquete del servidor multicast
22             ms.receive(paquete);
23             msg = new String(paquete.getData());
24             System.out.println("Recibo: " + msg.trim());
25         }
26
27         // Abandonamos grupo
28         ms.leaveGroup(grupo);
29
30         // Cerramos recursos
31         ms.close();
32         System.out.println("Socket cerrado...");
33     }
34 }

```

javi



mezcla de sockets en una app

Ya hemos visto todo el abanico de posibilidades que tenemos para comunicar dos procesos en red.

A partir de este momento, en nuestras aplicaciones no sólo tenemos que elegir uno de ellos, sino que podemos tener varios sockets, de diferente tipo, para comunicar los clientes y los servidores.

Se trata ahora de analizar en qué situación es más conveniente un tipo que otro y usarlo. Podemos ayudarnos de la creación de hilos que estén "especializados" en el envío y/o recepción de información de un socket, permitiendo que se intercambien varios mensajes a la vez.